

FFmpeg 时间基 (time_base) 与时间戳 (PTS/DTS) 深度解析报告

生成时间: 2026-06-24 基于 FFmpeg 编解码实践中关于时间基与时间戳的讨论整理

目录

- AVRational 结构体基础
- 三层时间基架构
- 时间基与时间戳的核心关系
- 视频时间基详解
- 音频时间基详解
- 时间基转换实践
- 常见问题与陷阱
- 总结

1. AVRational 结构体基础

1.1 定义

```
typedef struct AVRational {  
    int num; // 分子 (numerator)  
    int den; // 分母 (denominator)  
} AVRational;
```

AVRational 是 FFmpeg 中表示有理数的结构体, 用于精确表示分数值, 避免浮点精度问题。

1.2 为什么用有理数不用浮点数?

优势	说明
精度无损	1/90000 用浮点数是无限循环小数, 有理数精确表示
避免累积误差	音视频处理中时间戳计算频繁, 浮点误差会累积导致音视频不同步
编解码器兼容	大多数编解码器规范本身就使用有理数时间基

1.3 核心公式

```
实际时间 (秒) = 时间戳 × time_base  
              = 时间戳 × (num / den)  
  
时间戳 = 实际时间 (秒) / time_base  
        = 实际时间 (秒) × (den / num)
```

1.4 常用辅助函数

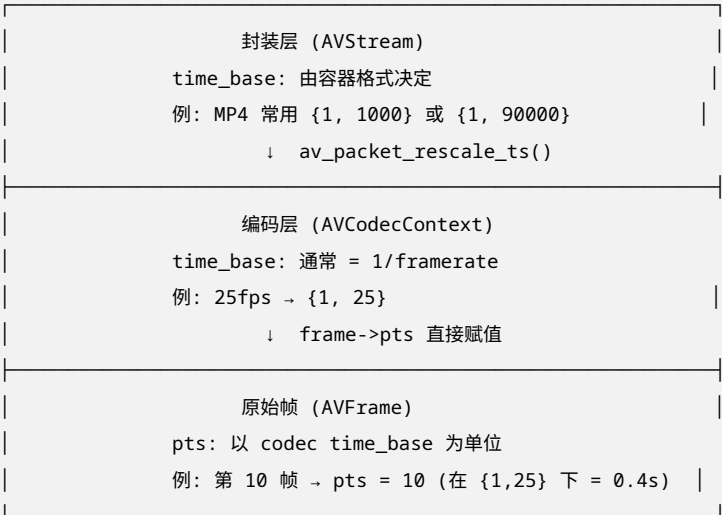
函数	作用
av_q2d(AVRational a)	将有理数转为 double: num / (double)den

Table 2 – continued

函数	作用
av_d2q(double d, int max)	将 double 转为有理数
av_rescale_q(int64_t a, AVRational bq, AVRational cq)	时间基转换的核心函数
av_compare_ts(...)	比较不同时间基下的时间戳
av_add_stable(...)	稳定地递增时间戳

2. 三层时间基架构

FFmpeg 编解码涉及 三个不同层面的时间基，它们之间必须正确转换：

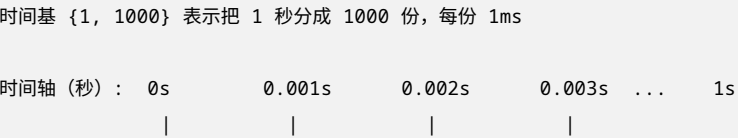


2.1 三层对比

层级	结构体	字段	谁设置	视频典型值	音频典型值	作用
原始层	AVFrame	pts	用户/解码器	0,1,2...	0,1024,2048...	逻辑时间戳
编码层	AVCodecContext	time_base	用户（编码）/编码器（解码）	{1,25}	{1,48000}	编码器工作基准
封装层	AVStream	time_base	容器/复用器	{1,1000}	{1,1000}	文件存储格式

3. 时间基与时间戳的核心关系

3.1 时间基的本质：一把” 尺子”



时间戳:	0	1	2	3	...	1000
时间基 {1, 25}	表示把 1 秒分成 25 份, 每份 40ms (一帧)					
时间轴 (秒):	0s	0.04s	0.08s	0.12s	...	1s
时间戳:	0	1	2	3	...	25

3.2 核心原则

时间戳是” 多少个单位”, time_base 是” 每个单位多少秒”, 两者相乘得到实际时间。不同层使用不同 time_base, 通过 av_rescale_q() 转换时间戳, 保证实际时间不变。

3.3 时间基转换公式

```
// 将时间戳从 time_base A 转换到 time_base B
// 公式: new_pts = old_pts × (A.num / A.den) / (B.num / B.den)
//          = old_pts × A.num × B.den / (A.den × B.num)

// FFmpeg 内部实现 (av_rescale_q) :
int64_t new_pts = av_rescale_q(old_pts, tb_a, tb_b);
```

4. 视频时间基详解

4.1 视频流中的 time_base

```
AVStream *video_stream = fmt_ctx->streams[video_index];

// 获取视频流的时间基 (解码时由 libavformat 设置)
AVRational video_tb = video_stream->time_base;
// 常见值: {1, 90000} 或 {1, 1000}

// 将时间戳 (pts/dts) 转换为秒
double seconds = av_q2d(video_tb) * video_stream->pts;

// 或将秒转换为时间戳
int64_t pts = seconds / av_q2d(video_tb);
```

4.2 编码时设置 time_base

```
// 【编码前】设置期望的时间基 (hint)
st->time_base = (AVRational){1, 25}; // 提示 muxer: 我希望用 1/25 秒为单位

// 编码器上下文的时间基通常设置为帧率的倒数
codec_ctx->time_base = (AVRational){1, 25}; // 25fps

// 调用 write_header 后, muxer 可能覆盖 st->time_base
avformat_write_header(oc, NULL);
```

```
// 【编码后】必须使用 st->time_base 来计算时间戳！
// 因为 muxer 可能已经修改了它
AVRational actual_tb = st->time_base;
```

4.3 视频帧 PTS 计算

```
// 假设编码第 N 帧，帧率为 25fps
int frame_number = 100;

// 方法 1：直接用时间基计算
AVRational tb = st->time_base; // 比如 {1, 25}
pkt->pts = frame_number * tb.den / tb.num / 25;
// 简化：若 tb={1,25}, pts 直接等于 frame_number

// 方法 2：使用 FFmpeg 辅助函数
int64_t pts_in_stream_tb = av_rescale_q(
    frame_number,          // 值
    (AVRational){1, 25},   // 源时间基（帧率）
    st->time_base           // 目标时间基（流时间基）
);
```

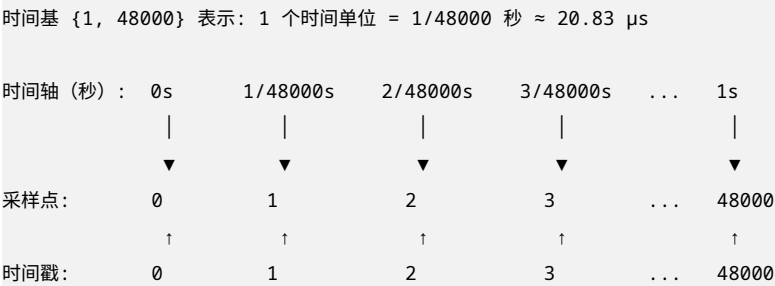
4.4 视频时间基对照表

场景	典型 time_base	含义	1 个时间单位
MP4 视频流	{1, 1000} 或 {1, 90000}	毫秒或 90kHz	1ms 或 ~11 s
FLV 视频流	{1, 1000}	毫秒	1ms
MPEG-TS	{1, 90000}	90kHz (ISO 标准)	~11.1 s
25fps 编码器	{1, 25}	帧间隔	40ms
30fps 编码器	{1, 30}	帧间隔	~33.3ms

5. 音频时间基详解

5.1 音频时间基的特殊性

音频与视频不同——音频的时间戳通常直接等于采样点序号 (sample index)。



关键发现：对于 48kHz 音频，时间戳 = 采样点序号！

5.2 音频帧 PTS 计算

```
// AAC 编码器通常一次处理 1024 个采样点
int samples_per_frame = 1024;
int sample_rate = 48000;

// 第 0 帧: 包含采样点 0 ~ 1023
// 播放时间: 0 ~ 1023/48000 秒
frame->pts = 0; // 时间戳 = 0

// 第 1 帧: 包含采样点 1024 ~ 2047
// 播放时间: 1024/48000 ~ 2047/48000 秒
frame->pts = 1024; // 时间戳 = 1024 (第一个采样点的序号)

// 第 2 帧
frame->pts = 2048;

// 通用公式
frame->pts = frame_number * samples_per_frame;
```

5.3 音频时间基对照表

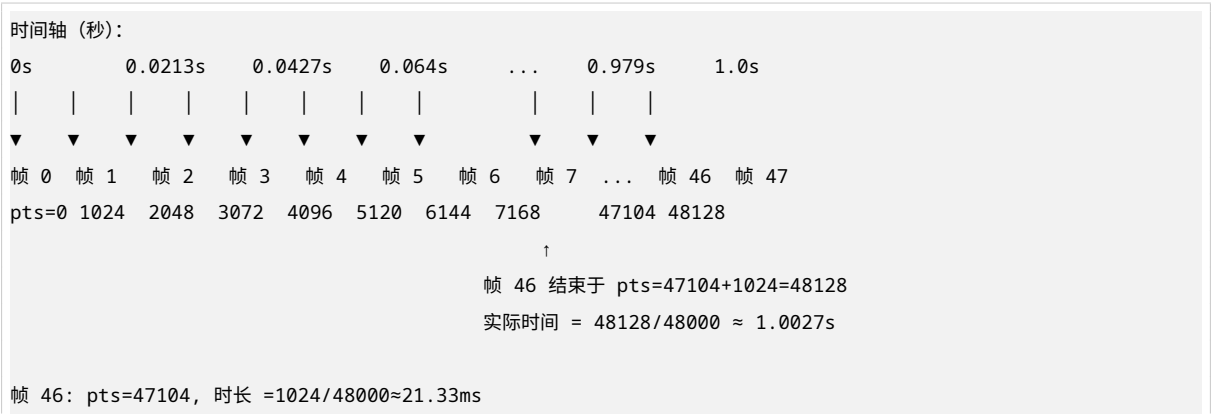
场景	典型 time_base	含义	1 个时间单位
音频 48kHz	{1, 48000}	采样周期	~20.8 s
音频 44.1kHz	{1, 44100}	采样周期	~22.7 s
MP3	{1, 44100} 或 {1, 90000}	采样周期或 90kHz	~22.7 s 或 ~11.1 s
微秒 (通用)	{1, 1000000}	微秒	1 s

5.4 音频帧边界与整秒不对齐问题

```
每帧 1024 采样点, pts 递增序列:
0, 1024, 2048, 3072, 4096, 5120, 6144, 7168, 8192, 9216, 10240...

问: 能否精确到达 48000 (1 秒)?
48000 ÷ 1024 = 46.875 ← 不是整数!
```

结论: pts 永远不会精确等于 48000, 意味着音频帧边界与整秒时刻不对齐。



覆盖时间范围: $47104/48000 \approx 0.9813s \sim 48128/48000 \approx 1.0027s$

1 秒时刻落在帧 46 的中间!

5.5 解决方案

```
// 方案 1: 接受不对齐 (实际做法)
// 大多数编码器/复用器自动处理, 无需干预
// 时间戳保持 1024 递增, 容器记录每帧实际时长
pkt->duration = 1024; // 以 time_base 为单位

// 方案 2: 理解"帧时长" vs "时间戳"
// • pts = 帧的起始采样点序号
// • duration = 帧包含的采样点数
// • 结束时刻 = (pts + duration) / 48000
// 1 秒时刻只是某帧的中间点, 不是帧边界, 这是完全正常的!
```

6. 时间基转换实践

6.1 编码完整流程

```
// ===== 第 1 层: 原始帧 =====
AVFrame *frame = av_frame_alloc();
frame->pts = frame_number; // 视频: 0,1,2... 音频: 0,1024,2048...

// ===== 第 2 层: 编码器 =====
// 设置编码器时间基
codec_ctx->time_base = (AVRational){1, 25}; // 视频
// codec_ctx->time_base = (AVRational){1, 48000}; // 音频

avcodec_send_frame(codec_ctx, frame);

// ===== 第 2 层 → 第 3 层: 转换 =====
while (avcodec_receive_packet(codec_ctx, pkt) == 0) {
    // 关键! 编码器输出是 codec time_base
    // 必须转换到 stream time_base 才能写入文件
    av_packet_rescale_ts(pkt,
                          codec_ctx->time_base, // 从编码器
                          stream->time_base);   // 到流

    av_interleaved_write_frame(fmt_ctx, pkt);
}
```

6.2 解码完整流程

```
// ===== 第 3 层: 读取文件 =====
while (av_read_frame(fmt_ctx, pkt) >= 0) {
    // pkt->pts 在 stream time_base 下

    // ===== 第 3 层 → 第 2 层: 转换 =====
```

```

    av_packet_rescale_ts(pkt,
                        stream->time_base,    // 从流
                        codec_ctx->time_base); // 到编码器

    avcodec_send_packet(codec_ctx, pkt);

    // ===== 第 2 层 → 第 1 层: 输出 =====
    while (avcodec_receive_frame(codec_ctx, frame) == 0) {
        // frame->pts 在 codec time_base 下
        double seconds = frame->pts * av_q2d(codec_ctx->time_base);
    }
}

```

6.3 编码时的完整时间戳处理

```

// 假设: 25fps 视频, 输出 MP4 (stream time_base 通常为 {1, 1000})

AVRational codec_tb = {1, 25};    // 编码器时间基
AVRational stream_tb = {1, 1000}; // 流时间基 (avformat_write_header 后确定)

// 编码第 i 帧 (从 0 开始)
for (int i = 0; i < 250; i++) {

    // === Step 1: 设置 AVFrame 的 pts ===
    // frame->pts 以 codec_ctx->time_base 为单位
    // 第 i 帧的时间 = i / 25 秒
    // 所以 pts = i / 25 / (1/25) = i
    frame->pts = i; // 直接等于帧序号!

    // 发送帧到编码器
    avcodec_send_frame(codec_ctx, frame);

    // 接收编码后的包
    while (avcodec_receive_packet(codec_ctx, pkt) == 0) {

        // === Step 2: 转换时间基 ===
        // 编码器输出的 pkt->pts 在 codec_tb 下
        // 需要转换到 stream_tb 才能写入文件
        //
        // 转换:  $\text{pkt->pts} \times (1/25) / (1/1000)$ 
        //      =  $\text{pkt->pts} \times 1000 / 25$ 
        //      =  $i \times 40$ 
        av_packet_rescale_ts(pkt, codec_tb, stream_tb);

        // 现在 pkt->pts = i * 40
        // 第 10 帧: pts = 400, 实际时间 =  $400/1000 = 0.4s$ 
        // 第 10 帧实际时间也应是  $10/25 = 0.4s$ 

        pkt->stream_index = video_st->index;
        av_interleaved_write_frame(fmt_ctx, pkt);
    }
}

```

```
}
```

6.4 解码时的完整时间戳处理

```
// 读取 packet (时间戳在 stream time_base 下)
while (av_read_frame(fmt_ctx, pkt) >= 0) {
    if (pkt->stream_index == video_idx) {

        // === 转换到编码器时间基 ===
        // 因为解码器期望的时间基是 codec_ctx->time_base
        av_packet_rescale_ts(pkt, stream_tb, codec_tb);

        // 发送 packet 到解码器
        avcodec_send_packet(codec_ctx, pkt);

        // 接收解码后的帧
        while (avcodec_receive_frame(codec_ctx, frame) == 0) {

            // frame->pts 现在以 codec_tb 为单位
            // 转换为秒用于显示/处理
            double display_time = frame->pts * av_q2d(codec_tb);

            printf("Frame pts: %ld, time: %.3f s\n",
                   frame->pts, display_time);
        }
    }
    av_packet_unref(pkt);
}
```

7. 常见问题与陷阱

7.1 常见陷阱

陷阱	说明
✗ 混用不同的时间基	codec_ctx->time_base 和 st->time_base 可能不同，必须用 av_rescale_q 转换
✗ 编码前假设 st->time_base 不变	avformat_write_header() 后 muxer 可能覆盖它
✗ 直接用浮点数计算 pts	会导致音视频不同步
✗ 忽略不同流的时间基差异	视频可能是 {1,90000}, 音频可能是 {1,44100}

7.2 为什么不同层用不同时间基？

问题：如果统一用 {1, 1000}（毫秒）表示 25fps 视频？

```
第 0 帧: 0ms   → pts = 0
第 1 帧: 40ms  → pts = 40
第 2 帧: 80ms  → pts = 80
...
```


问题：帧间隔是 40ms，但时间戳只能取整数毫秒，所以 pts 必须是 40 的倍数。
如果某帧需要 pts=50（表示 50ms），在 25fps 下不存在这个精确时间点！

- 解决方案：
- 编码器用 {1, 25}：每帧 pts 递增 1，完美对齐
 - 复用到文件时转换为 {1, 1000}：pts = 0, 40, 80, 120...
 - 播放时再从 {1, 1000} 转换回来，保持精度

8. 总结

8.1 核心要点

1. **AVRational** = 分子/分母，用于精确表示分数
2. **time_base** 是时间戳的”货币单位”，所有时间戳都是这个单位的整数倍
3. 解码时：time_base 由 libavformat 设置，直接用 av_q2d() 转秒
4. 编码时：先设置 hint，但 avformat_write_header() 后必须用实际的 st->time_base
5. 跨时间基转换：永远使用 av_rescale_q()，不要手动做乘除法

8.2 视频 vs 音频时间基对比

	视频 (25fps)	音频 (48kHz, 1024/frame)
time_base	{1, 25}	{1, 48000}
时间单位	40 ms (一帧)	~20.83 s (一个采样点)
pts 含义	帧序号	采样点序号
pts 递增	每帧 +1	每帧 +1024 (或每采样点 +1)
精度	粗 (40ms)	极细 (~21 s)
整秒对齐	✅ 精确 (25 整除)	❌ 不对齐 (48000 不整除 1024)

8.3 一句话总结

时间戳是”多少个单位”，time_base 是”每个单位多少秒”，两者相乘得到实际时间。不同层使用不同 time_base，通过 av_rescale_q() 转换时间戳，保证实际时间不变。

音频时间基 {1, 48000} 下，时间戳就是采样点序号。pts = N 意味着”从第 N 个采样点开始播放”，实际时间 = N/48000 秒。

报告结束